PHP Object-Oriented Programming (OOP) & MVC Cheat Sheet

---

1. Basics of OOP

- **Class**: Blueprint for objects.
- **Object**: Instance of a class.
- **Properties**: Variables inside a class.
- **Methods**: Functions inside a class.
- **$this**: Refers to the current object.

Example

```
class Car {
    public $brand;
    public $color;

    public function __construct($brand, $color) {
        $this->brand = $brand;
        $this->color = $color;
    }

    public function drive() {
        echo "The $this->color $this->brand is driving!";
    }
}
$myCar = new Car("Toyota", "red");
$myCar->drive();
```

---

2. Visibility (Access Modifiers)

Keyword	Accessible From
public	anywhere
protected	class + subclasses
private	class only

---

3. Inheritance

```
class Vehicle {
    public function start() { echo "Starting vehicle..."; }
}
class Car extends Vehicle {
```

```
    public function honk() { echo "Beep!"; }
}
```

4. Static Members

```
class MathHelper {
    public static $pi = 3.14;
    public static function square($x) { return $x * $x; }
}
```

5. Interfaces

```
interface Animal { public function makeSound(); }
class Dog implements Animal {
    public function makeSound() { echo "Woof!"; }
}
```

6. Abstract Classes

```
abstract class Shape { abstract public function area(); }
class Circle extends Shape { public function area() { echo "πr²"; } }
```

7. Traits

```
trait Logger { public function log($msg) { echo "[LOG]: $msg"; } }
class App { use Logger; }
```

---

8. Getters & Setters

- Control access to private/protected properties.
- Can include validation or logic.

Example

```
class User {
    private $name;
    public function setName($name) {
        if(strlen($name)<3) throw new Exception("Name too short!");
        $this->name = $name;
    }
```

```
    public function getName() { return $this->name; }
}
```

Magic Methods

```
public function __get($name) { return $this->data[$name] ?? null; }
public function __set($name,$value) { $this->data[$name] = $value; }
```

---

9. Constructor + Setters (Best Practice)

```
public function __construct($name,$age) {
    $this->setName($name);
    $this->setAge($age);
}
```

- Use constructor to initialize - Call setters inside constructor for validation

---

10. Public Attributes

- Can skip getters/setters if `public`.
- Not recommended for production due to lack of validation.
- Use private + getters/setters for safety and maintainability.

---

11. MVC Structure

- **Model**: Handles data, DB interaction (`User.php`).
- **View**: Handles HTML/CSS (`user_view.php`).
- **Controller**: Handles logic, user input (`UserController.php`).

Example

```
// Model/User.php
class User { private $name; public function __construct($name){$this-
>name=$name;} public function getName(){return $this->name;} }

// Controller/UserController.php
require_once '../model/User.php';
class UserController { public function showUser(){ $user = new User("Alex");
include '../view/user_view.php'; } }
```

```
// View/user_view.php
<h1>Hello, <?= htmlspecialchars($user->getName()); ?>!</h1>
```

- Controller creates the Model and passes it to View - View displays the data

---

12. Best Practices Summary

Scenario	Constructor	Setters	Notes
Immutable object	✅	❌	Use for constants/configs
Step-by-step filling	❌	✅	Forms, interactive data
Full validation	✅	✅	Constructor calls setters, safest approach